Event AI Planner

Multi-Agent AI System for Event Planning

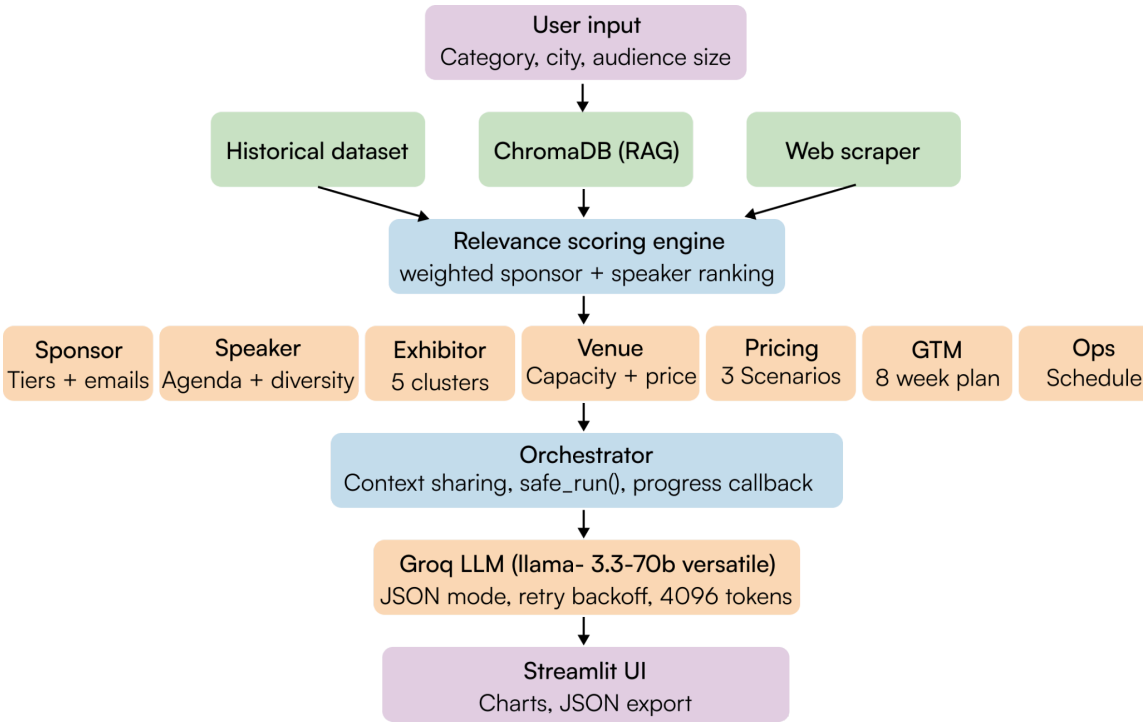
1. Introduction

Conferences and large-scale events involve multiple interconnected components such as sponsors, speakers, exhibitors, venues, pricing strategies, and marketing. Traditionally, organizing such events requires manual coordination across fragmented tools, making the process inefficient and difficult to scale.

This project presents an AI-powered multi-agent system that automates end-to-end conference planning by leveraging historical data, semantic search, and intelligent agents

---

2. System Overview



The system follows a modular multi-agent architecture, where each agent specializes in one part of the event lifecycle.

Key Capabilities:

- Autonomous decision-making using AI agents
- Context sharing across agents

- Data-driven recommendations (not generic outputs)
- End-to-end event planning in a single pipeline

---

3. Architecture Design

3.1 Multi-Agent Architecture

The system consists of the following agents:

Agent	Function
Sponsor Agent	Recommends and ranks sponsors
Speaker Agent	Suggests speakers + agenda
Exhibitor Agent	Identifies and clusters exhibitors
Venue Agent	Recommends venues based on constraints
Pricing Agent	Predicts pricing and attendance
GTM Agent	Builds marketing strategy
Event Ops Agent	Generates schedule + conflict detection

3.2 Orchestrator (Core Controller)

The orchestrator acts as the central coordination engine.

Responsibilities:

- Agent execution sequencing
- Error handling (`_safe_run`)
- Context sharing between agents

Context Sharing Examples:

- Speaker count → Pricing
- Sponsors → GTM
- Pricing → Event Ops

---

4. Data Architecture

4.1 Data Sources

- Historical dataset (2025–2026 events)
- Web scraping (Eventbrite, Luma)
- Predefined community datasets

4.2 Components

Data Loader

- Loads and preprocesses event data
- Supports filtering by category and geography

Embeddings Layer (RAG)

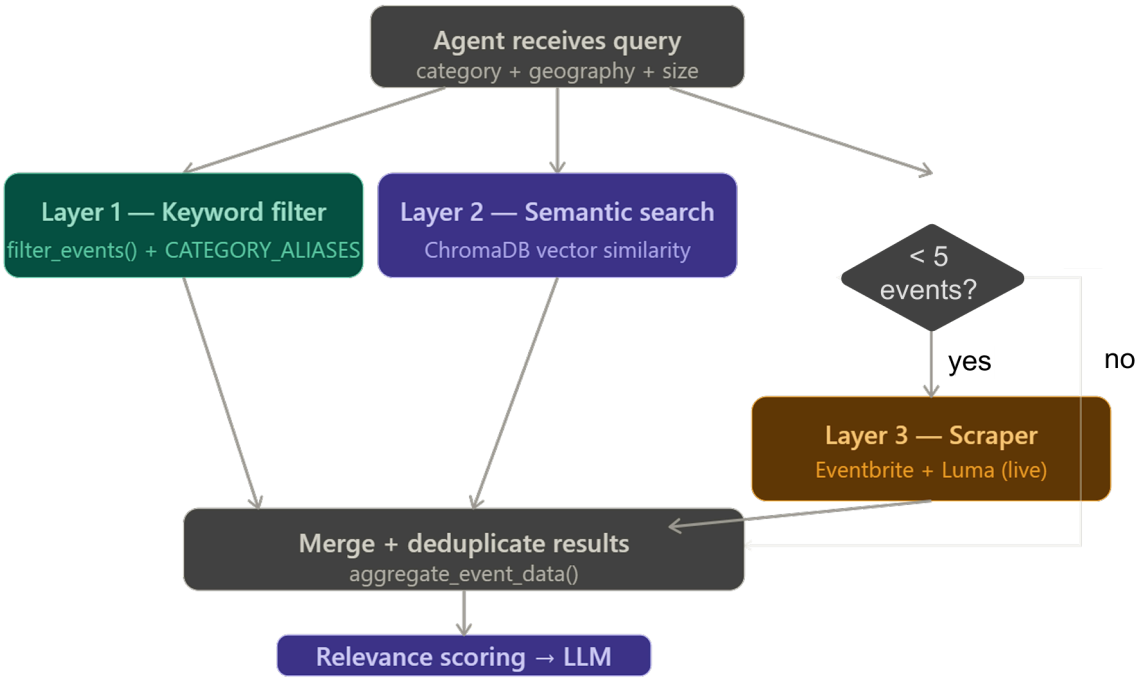
- Converts events into vector representations
- Enables semantic similarity search

Web Scraper

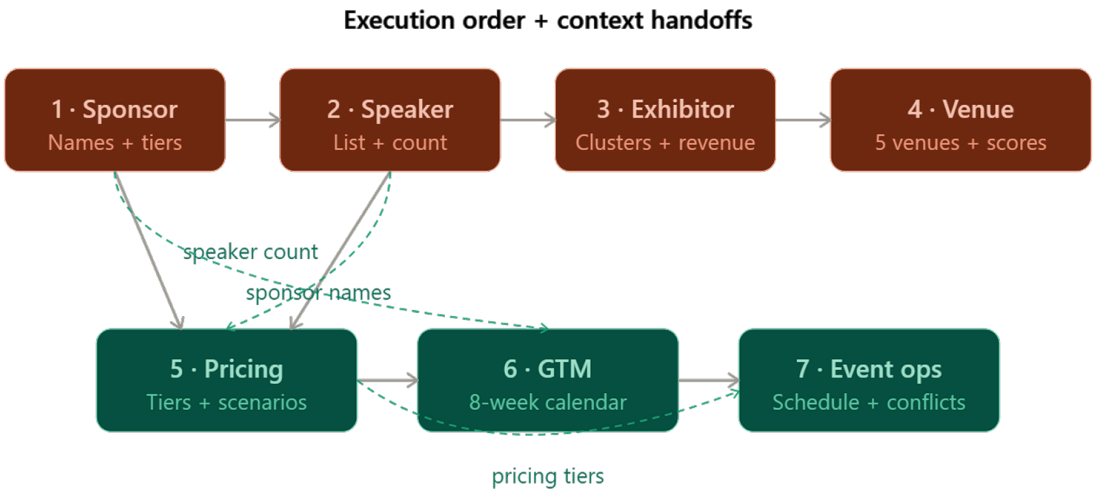
- Fetches real-time event data when historical data is insufficient

Ranking Engine

- Scores entities using weighted formulas



5. Agent Design



Agent	Primary Function	Key Output
Sponsor Agent	Recommend and rank sponsors	Tiered sponsor list + outreach emails
Speaker Agent	Suggest speakers + agenda	Speaker lineup + keynote themes + diversity metrics
Exhibitor Agent	Identify and cluster exhibitors	Clustered exhibitor list + booth pricing
Venue Agent	Recommend venues	Scored venue list + negotiation tips
Pricing Agent	Predict pricing + financials	Pricing tiers + revenue scenarios + break-even
GTM Agent	Build marketing strategy	8-week GTM calendar + outreach templates
Event Ops Agent	Build schedule + detect conflicts	Conflict-free agenda + resource plan

6. Orchestrator Design

The **Orchestrator** is the central coordination engine of the system. It is responsible for managing the lifecycle of the multi-agent pipeline, ensuring that all agents execute in the correct order, share relevant context, and produce a coherent final output.

1. Agent Execution Management

The orchestrator defines and controls the **execution sequence of agents**. Since several agents depend on outputs from previous agents, a strict order is maintained.

- Ensures **deterministic workflow execution**
- Prevents missing dependencies between agents
- Guarantees completeness of the final plan

2. Dependency Resolution

Each agent is not independent — it relies on outputs from earlier stages.

Examples:

- **Pricing Agent** requires:
    - Expected number of speakers (from Speaker Agent)
    - Estimated audience (input)
- **GTM Agent** uses:
    - Sponsor list (from Sponsor Agent)
    - Event category (input)
- **Event Ops Agent** depends on:
    - Speaker list (Speaker Agent)
    - Pricing tiers (Pricing Agent)

The orchestrator ensures that: “No agent runs without the required upstream context”

3. Context Sharing Mechanism

The orchestrator maintains a **shared context object (state dictionary)** that stores outputs from each agent.

Each agent:

- Reads required inputs from this context
- Writes its output back into it

This enables **loosely coupled but coordinated agents**.

4. Fault Tolerance & Error Handling

To ensure robustness, each agent call is wrapped inside a `_safe_run()` function.

Purpose:

- Prevents system crash if one agent fails
- Logs errors for debugging
- Allows pipeline to continue with partial results

5. Execution sequence

Sponsor → Speaker → Exhibitor → Venue → Pricing → GTM → Event Ops

Why this order?

- **Sponsors first** → needed for partnerships + GTM
- **Speakers next** → defines agenda + scale
- **Exhibitors & Venue** → logistical planning
- **Pricing after data is ready** → ensures realistic modeling
- **GTM after sponsors** → co-marketing strategies
- **Event Ops last** → needs all prior outputs

---

7. AI & LLM Integration

The system integrates a **Large Language Model (LLM)** as the core reasoning engine that powers decision-making across all agents. Instead of using the LLM independently within each module, the architecture introduces a **centralized LLM interface (`tools/llm.py`)**, ensuring consistency, reliability, and scalability across the entire multi-agent system.

1. Centralized LLM Interface

All agents interact with the LLM through a single function:

call_llm(prompt, system_prompt=None)

This design ensures:

- Consistent prompting across agents
- Easier debugging and maintenance
- Reusability of logic

2. Structured Output (JSON Mode)

To ensure reliability, the system enforces **JSON-formatted responses** from the LLM.

- Prevents unstructured outputs
- Makes responses easy to parse
- Enables smooth integration with UI and other agents

3. Prompt Design Strategy

Each agent sends a **structured prompt** containing:

- Role definition (e.g., sponsor expert)
- Relevant context (ranked data, filters)
- Clear instructions and output format

Key idea:

> The LLM is used for reasoning and generation, while data processing is handled by the backend.

4. Role of LLM in the System

The LLM is responsible for:

- Generating recommendations (sponsors, speakers, venues)
- Creating content (emails, strategies, agendas)
- Interpreting ranked data into actionable insights

5. Hybrid Intelligence Approach

The system combines:

- **Backend logic** → filtering, ranking, data handling
- **LLM** → reasoning, synthesis, content generation

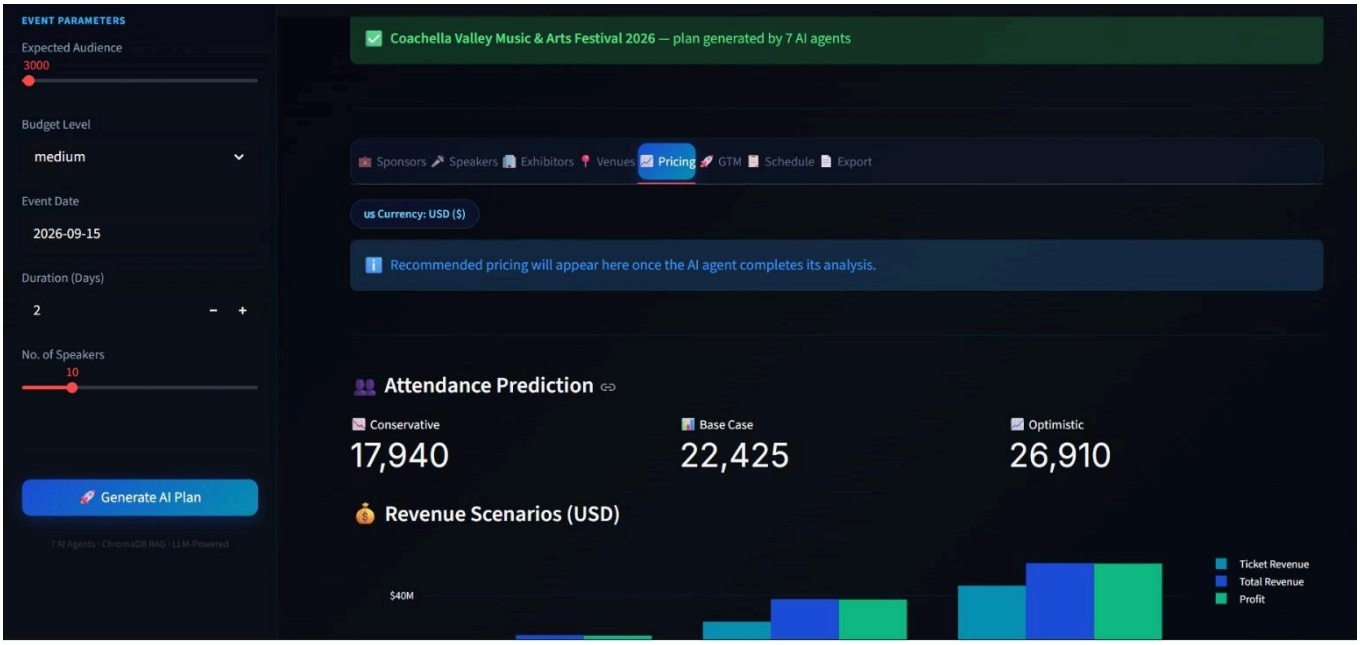
This reduces hallucination and improves accuracy.

The LLM acts as a reasoning layer on top of structured data, enabling intelligent and context-aware decision-making across the multi-agent system.

---

8. Results

System Output

When a user inputs a conference name, category, geography, and target audience size, the system autonomously generates a complete event plan across 8 tabs in the Streamlit dashboard:

Tab	Output
Sponsors	Ranked sponsor list (Platinum / Gold / Silver / Bronze), priority scores, customized proposals, outreach emails
Speakers	Recommended speaker lineup, talk titles, session types (keynote/panel/workshop), keynote themes
Exhibitors	Clustered exhibitor list (Startup, Enterprise, Tools, Research, Individual) with booth pricing
Venues	Top venues with capacity, estimated pricing, scoring breakdown, negotiation tips
Pricing	Ticket tiers (Early Bird, Standard, VIP), conversion rates, forecasts, revenue scenarios, break-even
GTM Strategy	8-week GTM calendar, communities, messaging, hashtags, SEO keywords, outreach templates
Schedule	Multi-day, multi-track agenda with conflict detection (room/speaker clashes flagged)
Export	Complete plan as structured JSON for external tool integration